

NAME: NISHI CHOPDA
JEFF BEZOS
150096725128

Figure 1: ER Diagram of GreenCycle – Community Waste Collection & Recycling Scheduler (Manual Made)

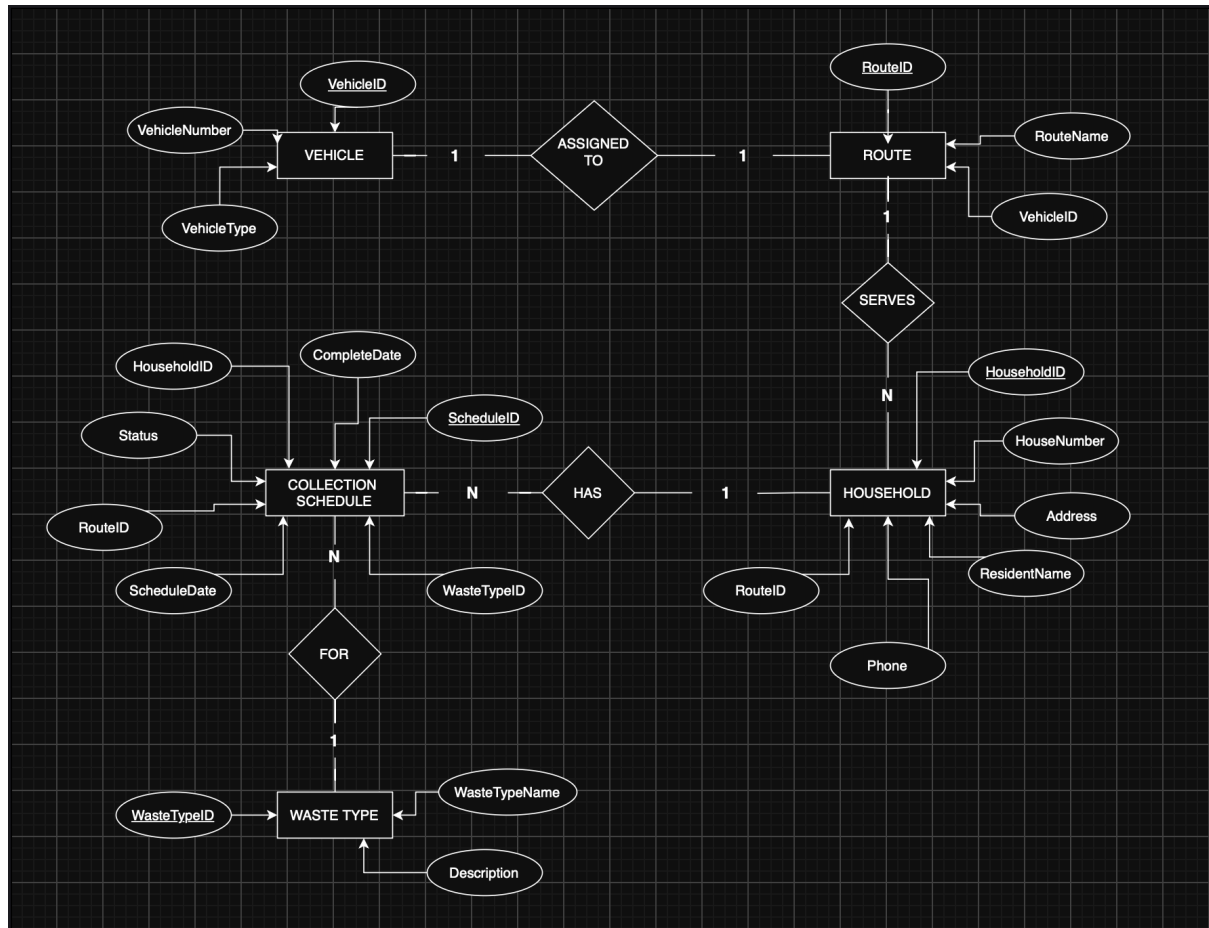
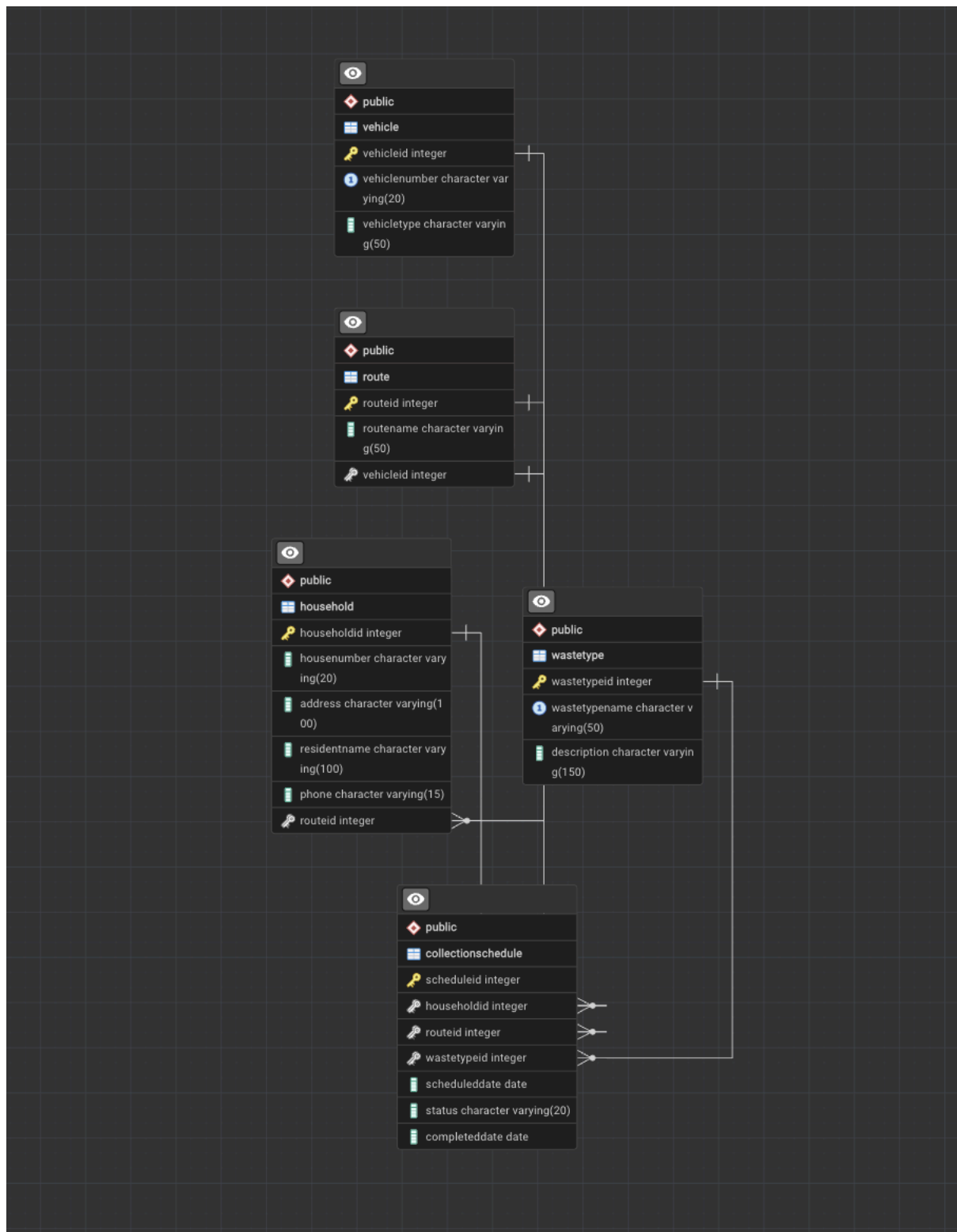


Figure 2: ER Diagram of GreenCycle – Community Waste Collection & Recycling Scheduler (PgAdmin Generated)



Schema Design

The ER diagram is converted into a relational schema consisting of five tables: Household, Route, Vehicle, WasteType, and CollectionSchedule. Each table has a primary key for unique identification, while foreign keys are used to establish relationships between the tables.

Relational Schema

```
HOUSEHOLD(  
    HouseholdID PK,  
    HouseNumber,  
    Address,  
    ResidentName,  
    Phone,  
    RouteID FK  
)
```

```
ROUTE(  
    RouteID PK,  
    RouteName,  
    VehicleID FK  
)
```

```
VEHICLE(  
    VehicleID PK,  
    VehicleNumber,  
    VehicleType  
)
```

```
WASTETYPE(  
    WasteTypeID PK,  
    WasteTypeName,  
    Description
```

)

COLLECTIONSCHEDULE(

ScheduleID PK,
HouseholdID FK,
RouteID FK,
WasteTypeID FK,
ScheduledDate,
Status,
CompletedDate

)

Normalization:

The GreenCycle database is normalized to reduce data redundancy and avoid insertion, update, and deletion anomalies. The tables satisfy 1NF by maintaining atomic attributes and avoiding repeating groups. The design satisfies 2NF because non-key attributes depend on the complete primary key. The tables are separated according to their entities, so Route, Vehicle, Household, and WasteType information is not unnecessarily repeated in CollectionSchedule. The design also avoids transitive dependencies between non-key attributes, supporting 3NF. RouteID is retained in CollectionSchedule as a foreign key to identify the route associated with each collection.

SQL Scripts / DDL

1. Create Vehicle Table

Theory:

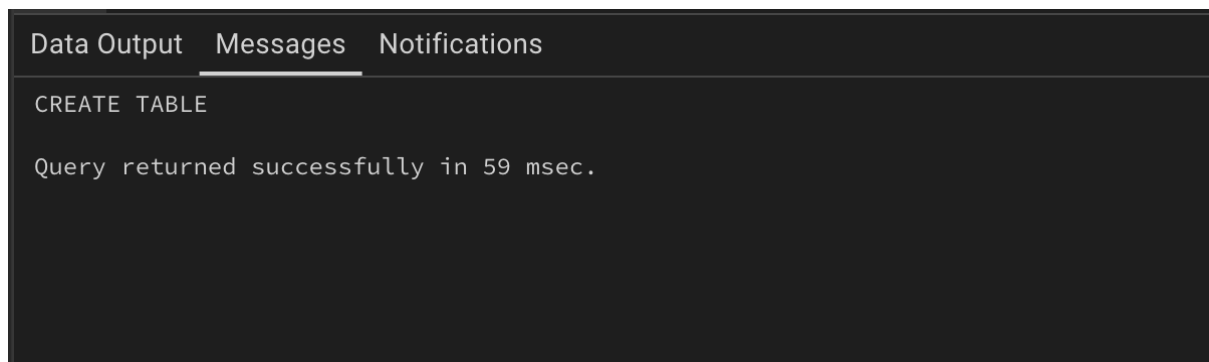
The Vehicle table stores information about the vehicles used for waste collection. VehicleID is the primary key used to uniquely identify each vehicle. VehicleNumber is defined as NOT NULL and UNIQUE to ensure that every vehicle has a

valid and distinct registration number. VehicleType stores the type of vehicle used for collection.

SQL Query:

```
CREATE TABLE Vehicle (  
    VehicleID INT PRIMARY KEY,  
    VehicleNumber VARCHAR(20) NOT NULL UNIQUE,  
    VehicleType VARCHAR(50) NOT NULL  
);
```

Result :



2. Create Route Table

The Route table stores the collection routes used by the waste collection service. RouteID is the primary key that uniquely identifies each route. RouteName stores the name of the route, while VehicleID is a foreign key that connects the route to its assigned vehicle. A UNIQUE constraint is applied to VehicleID because each vehicle is assigned to only one route in the designed 1:1 relationship.

SQL Query

```
CREATE TABLE Route (
```

```
RouteID INT PRIMARY KEY,  
RouteName VARCHAR(50) NOT NULL,  
VehicleID INT NOT NULL UNIQUE,  
FOREIGN KEY (VehicleID) REFERENCES Vehicle(VehicleID)  
);
```

Result:

Data Output	Messages	Notifications
<pre>CREATE TABLE Query returned successfully in 35 msec.</pre>		

3. Create WasteType Table

The WasteType table stores the different categories of waste collected by the service. WasteTypeID is the primary key used to uniquely identify each waste type. WasteTypeName stores the name of the waste category, while Description provides additional information about it.

SQL

```
CREATE TABLE WasteType (  
    WasteTypeID INT PRIMARY KEY,  
    WasteTypeName VARCHAR(50) NOT NULL UNIQUE,  
    Description VARCHAR(150)  
);
```

Result:

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 29 msec.

4. Create Household Table

The Household table stores information about the households registered for waste collection. HouseholdID is the primary key used to uniquely identify each household. The RouteID foreign key connects each household to its assigned collection route.

SQL Query

```
CREATE TABLE Household (  
    HouseholdID INT PRIMARY KEY,  
    HouseNumber VARCHAR(20) NOT NULL,  
    Address VARCHAR(100) NOT NULL,  
    ResidentName VARCHAR(100) NOT NULL,  
    Phone VARCHAR(15),  
    RouteID INT NOT NULL,  
    FOREIGN KEY (RouteID) REFERENCES Route(RouteID));
```

Result:

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 23 msec.

5. Create CollectionSchedule Table

The CollectionSchedule table stores the collection schedule for each household. ScheduleID is the primary key used to uniquely identify each schedule. HouseholdID, RouteID, and WasteTypeID are foreign keys that connect the schedule to the corresponding household, route, and waste type.

ScheduledDate stores the planned collection date, Status stores the current collection status, and CompletedDate stores the date when the collection was completed.

SQL Query

```
CREATE TABLE CollectionSchedule (  
    ScheduleID INT PRIMARY KEY,  
    HouseholdID INT NOT NULL,  
    RouteID INT NOT NULL,  
    WasteTypeID INT NOT NULL,  
    ScheduledDate DATE NOT NULL,  
    Status VARCHAR(20) NOT NULL,  
    CompletedDate DATE,  
    FOREIGN KEY (HouseholdID) REFERENCES  
Household(HouseholdID),  
    FOREIGN KEY (RouteID) REFERENCES Route(RouteID),
```

```
FOREIGN KEY (WasteTypeID) REFERENCES  
WasteType(WasteTypeID)  
);
```

Result:

Data Output	Messages	Notifications
-------------	----------	---------------

CREATE TABLE

Query returned successfully in 38 msec.

DML – Sample Data Insertion

1. Insert Sample Vehicle Records

Sample vehicle records are inserted into the Vehicle table to represent the vehicles used for community waste collection. Each vehicle has a unique VehicleID and registration number.

SQL Query

Inserting values:

```
INSERT INTO Vehicle (VehicleID, VehicleNumber, VehicleType)
VALUES
(101, 'MH01AB1234', 'Recycling Truck'),
(102, 'MH02CD5678', 'Waste Collection Truck'),
(103, 'MH03EF9012', 'Organic Waste Truck'),
(104, 'MH04GH3456', 'Recycling Truck');
```

Displaying values:

```
SELECT * FROM Vehicle;
```

Result:

Data Output Messages Notifications			
			
			SQL
	vehicleid [PK] integer 	vehiclenumber character varying (20) 	vehicletype character varying (50) 
1	101	MH01AB1234	Recycling Truck
2	102	MH02CD5678	Waste Collection Truck
3	103	MH03EF9012	Organic Waste Truck
4	104	MH04GH3456	Recycling Truck

2. Insert Sample Route Records

Sample collection routes are inserted into the Route table. Each route is assigned a vehicle using VehicleID. Since the Route table has a UNIQUE constraint on VehicleID, each vehicle can be assigned to only one route.

SQL

Inserting Values:

```
INSERT INTO Route (RouteID, RouteName, VehicleID)
VALUES
(201, 'Andheri Route', 101),
(202, 'Powai Route', 102),
(203, 'Bandra Route', 103),
(204, 'Borivali Route', 104);
```

Displaying the values:

```
SELECT * FROM Route;
```

Result:

Data Output

Messages

Notifications

≡+

SQL

	routeid [PK] integer	routename character varying (50)	vehicleid integer
1	201	Andheri Route	101
2	202	Powai Route	102
3	203	Bandra Route	103
4	204	Borivali Route	104

3. Insert Sample Waste Type Records

Sample waste categories are inserted into the WasteType table to represent the different types of waste collected by the service. Each waste type is identified using a unique WasteTypeID.

SQL Query

```
INSERT INTO WasteType (WasteTypeID, WasteTypeName,
Description)
VALUES
(301, 'Plastic', 'Plastic bottles, containers and packaging'),
(302, 'Paper', 'Newspapers, cardboard and other paper waste'),
(303, 'Organic', 'Food waste and other biodegradable material'),
(304, 'Glass', 'Glass bottles and containers');
```

Displaying value:

```
SELECT * FROM WasteType;
```

Result:

Data Output	Messages	Notifications
		
		
wastetypeid [PK] integer	wastetypename character varying (50)	description character varying (150)
1	301	Plastic
2	302	Paper
3	303	Organic
4	304	Glass

4. Insert Sample Household Records

Sample household records are inserted to represent households registered for waste collection. Each household is assigned to a valid collection route using RouteID, maintaining referential integrity between the Household and Route tables.

SQL Query

Inserting values:

```
INSERT INTO Household
(HouseholdID, HouseNumber, Address, ResidentName, Phone,
RouteID)
VALUES
(401, '12A', 'Andheri East', 'Nishi Chopda', '9876501201', 201),
(402, '25B', 'Powai', 'Nandeni Tiwari', '9876501202', 202),
(403, '8C', 'Bandra West', 'Yashika Saraf', '9876501203', 203),
(404, '16D', 'Borivali West', 'Akshara Tanted', '9876501204', 204),
(405, '21A', 'Andheri East', 'Vyom Shah', '9876501205', 201),
```

(406, '34B', 'Powai', 'Karan Dodia', '9876501206', 202),
(407, '7A', 'Bandra West', 'Prathamesh More', '9876501207', 203),
(408, '19C', 'Borivali West', 'Daksh Shrivastav', '9876501208', 204),
(409, '42B', 'Andheri East', 'Riya Mehta', '9876501209', 201),
(410, '11C', 'Powai', 'Arjun Patil', '9876501210', 202);

Displaying values:

```
SELECT * FROM Household;
```

Result:

	householdid [PK] integer	housetnumber character varying (20)	address character varying (100)	residentname character varying (100)	phone character varying (15)	routeid integer
1	401	12A	Andheri East	Nishi Chopda	9876501201	201
2	402	25B	Powai	Nandeni Tiwari	9876501202	202
3	403	8C	Bandra West	Yashika Saraf	9876501203	203
4	404	16D	Borivali West	Akshara Tanted	9876501204	204
5	405	21A	Andheri East	Vyom Shah	9876501205	201
6	406	34B	Powai	Karan Dodia	9876501206	202
7	407	7A	Bandra West	Prathamesh More	9876501207	203
8	408	19C	Borivali West	Daksh Shrivastav	9876501208	204
9	409	42B	Andheri East	Riya Mehta	9876501209	201
10	410	11C	Powai	Arjun Patil	9876501210	202

5. Insert Sample Collection Schedule Records

Sample collection schedule records are inserted to represent planned and completed waste collections for households. Each schedule is associated with a household, route, and waste type through foreign keys. The Status field indicates whether the collection is Scheduled, Completed, or Missed.

SQL

Inserting values:

```
INSERT INTO CollectionSchedule
(ScheduleID, HouseholdID, RouteID, WasteTypeID,
ScheduledDate, Status, CompletedDate)
VALUES
(501, 401, 201, 301, '2026-09-28', 'Completed', '2026-09-28'),
(502, 402, 202, 302, '2026-09-28', 'Completed', '2026-09-28'),
(503, 403, 203, 303, '2026-09-28', 'Completed', '2026-09-28'),
(504, 404, 204, 304, '2026-09-28', 'Completed', '2026-09-28'),
(505, 405, 201, 301, '2026-09-28', 'Completed', '2026-09-28'),
(506, 406, 202, 302, '2026-09-28', 'Completed', '2026-09-28'),
(507, 407, 203, 303, '2026-09-28', 'Completed', '2026-09-28'),
(508, 408, 204, 304, '2026-09-28', 'Completed', '2026-09-28'),
(509, 409, 201, 301, '2026-09-28', 'Completed', '2026-09-28');
```

Displaying values:

```
SELECT * FROM CollectionSchedule;
```

Result:

Data Output Messages Notifications							
	scheduleid [PK] integer	householdid integer	routeid integer	wastetypeid integer	scheduleddate date	status character varying (20)	completeddate date
1	501	401	201	301	2026-09-28	Completed	2026-09-28
2	502	402	202	302	2026-09-28	Completed	2026-09-28
3	503	403	203	303	2026-09-28	Completed	2026-09-28
4	504	404	204	304	2026-09-28	Completed	2026-09-28
5	505	405	201	301	2026-09-28	Completed	2026-09-28
6	506	406	202	302	2026-09-28	Completed	2026-09-28
7	507	407	203	303	2026-09-28	Completed	2026-09-28
8	508	408	204	304	2026-09-28	Completed	2026-09-28
9	509	409	201	301	2026-09-28	Completed	2026-09-28

Query 1: Find Households Missed in the Last Scheduled Collection

A LEFT JOIN is used to display all households, including those that do not have a matching collection record. The NULL check identifies households for which no completed collection record exists for the scheduled date.

Query

```
SELECT
    h.HouseholdID,
    h.ResidentName,
    h.Address,
    h.RouteID
FROM Household h
LEFT JOIN CollectionSchedule cs
    ON h.HouseholdID = cs.HouseholdID
    AND cs.ScheduledDate = '2026-09-28'
    AND cs.Status = 'Completed'
WHERE cs.ScheduleID IS NULL;
```

Result:

Data Output Messages Notifications				
	householdid [PK] integer	residentname character varying (100)	address character varying (100)	routeid integer
1	410	Arjun Patil	Powai	202

Query 2: Count Households Served per Route

This query calculates the number of households served by each route during the collection period. The COUNT aggregate function is used with GROUP BY to calculate the number of completed collections for each route.

SQL Query

```
SELECT
    r.RouteID,
    r.RouteName,
    COUNT(cs.HouseholdID) AS HouseholdsServed
FROM Route r
LEFT JOIN CollectionSchedule cs
    ON r.RouteID = cs.RouteID
    AND cs.Status = 'Completed'
    AND cs.ScheduledDate = '2026-09-28'
GROUP BY r.RouteID, r.RouteName
ORDER BY r.RouteID;
```

Result:

Data Output Messages Notifications				
			SQL	
	routeid [PK] integer	routename character varying (50)	householdsserved bigint	
1	201	Andheri Route	3	
2	202	Powai Route	2	
3	203	Bandra Route	2	
4	204	Borivali Route	2	

Query 3: Find Households on Highly Served Routes

A subquery is used to identify routes that completed more than two household collections on the scheduled date. The outer query then retrieves the households assigned to those routes. The GROUP BY and HAVING clauses inside the subquery are used to filter routes based on their completed collection count.

SQL Query

```
SELECT
    HouseholdID,
    ResidentName,
    Address,
    RouteID
FROM Household
WHERE RouteID IN (
    SELECT RouteID
    FROM CollectionSchedule
    WHERE Status = 'Completed'
    AND ScheduledDate = '2026-09-28'
    GROUP BY RouteID
    HAVING COUNT(*) > 2
)
```

```
HAVING COUNT(HouseholdID) > 2
)
ORDER BY HouseholdID;
```

Result:

Data Output Messages Notifications				
         SQL				
	householdid [PK] integer 	residentname character varying (100) 	address character varying (100) 	routeid integer 
1	401	Nishi Chopda	Andheri East	201
2	405	Vyom Shah	Andheri East	201
3	409	Riya Mehta	Andheri East	201

Trigger Implementation

Automatic Next Collection Scheduling

A trigger is implemented in the GreenCycle database to automatically generate the next collection schedule whenever a household's collection is marked as completed. The trigger checks for a change in the collection status to Completed and automatically inserts a new record into the CollectionSchedule

table. The new schedule is created for the same household, route, and waste type, with the scheduled date set to seven days after the current date and its status set to Scheduled.

1. Create Trigger Function

A PostgreSQL trigger function is created to define the actions that should be performed when a collection is marked as completed. The function inserts a new collection schedule using the household, route, and waste type information from the completed record. The next collection date is calculated using `CURRENT_DATE + 7`.

SQL Query

```
CREATE OR REPLACE FUNCTION generate_next_collection()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.Status = 'Completed'
        AND (OLD.Status IS DISTINCT FROM 'Completed') THEN

        INSERT INTO CollectionSchedule
        (ScheduleID, HouseholdID, RouteID, WasteTypeID,
        ScheduledDate, Status, CompletedDate)
        VALUES
        (
            (SELECT COALESCE(MAX(ScheduleID), 500) + 1
            FROM CollectionSchedule),
            NEW.HouseholdID,
            NEW.RouteID,
            NEW.WasteTypeID,
            CURRENT_DATE + 7,
            'Scheduled',
            NULL
        );
```

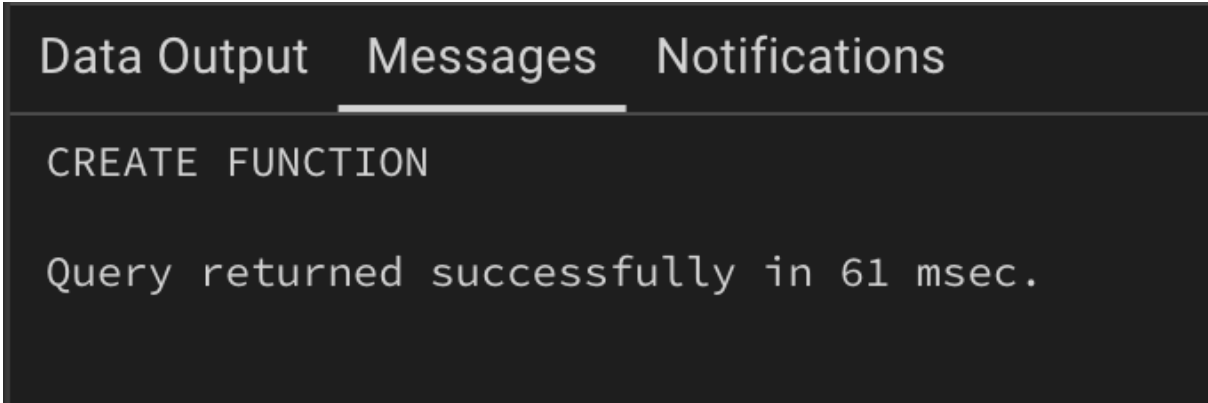
```
END IF;
```

```
RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

Result:



Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 61 msec.

2. Create Trigger

The trigger is attached to the CollectionSchedule table and executes the generate_next_collection() function whenever the Status column is updated. The trigger operates for each affected row and automatically generates the next collection schedule when the status changes to Completed.

SQL Query

```
CREATE TRIGGER trg_generate_next_collection  
AFTER UPDATE OF Status ON CollectionSchedule  
FOR EACH ROW  
EXECUTE FUNCTION generate_next_collection();
```

Result:

Data Output	Messages	Notifications
CREATE TRIGGER		
Query returned successfully in 41 msec.		

3. Trigger Testing

The trigger is tested by first creating a scheduled collection for a household and then updating its status to Completed. After the update, the trigger automatically creates the household's next collection schedule seven days after the current date. For testing, HouseholdID 410 (Arjun Patil) is used because it did not have an earlier collection record in the sample data.

Step 1: Insert a Scheduled Collection

```
INSERT INTO CollectionSchedule  
(ScheduleID, HouseholdID, RouteID, WasteTypeID,  
ScheduledDate, Status, CompletedDate)  
VALUES  
(510, 410, 202, 303, CURRENT_DATE, 'Scheduled', NULL);
```

Result:

Data Output Messages Notifications

```
INSERT 0 1
```

```
Query returned successfully in 28 msec.
```

Step 2: Mark the Collection as Completed

```
UPDATE CollectionSchedule  
SET Status = 'Completed',  
    CompletedDate = CURRENT_DATE  
WHERE ScheduleID = 510;
```

When this query executes, the trigger automatically runs.

Result:

Data Output Messages Notifications

```
UPDATE 1
```

```
Query returned successfully in 33 msec.
```

Step 3: Verify Automatically Generated Schedule

```
SELECT *  
FROM CollectionSchedule  
WHERE HouseholdID = 410
```

ORDER BY ScheduleID;

Result:

Data OutputMessagesNotifications

≡+

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

Showing rows: 1 to 2✎

⌵

	scheduleid [PK] integer✎	householdid integer✎	routeid integer✎	wastetypeid integer✎	scheduleddate date✎	status character varying (20)✎	completeddate date✎
1	510	410	202	303	2026-09-29	Completed	2026-09-29
2	511	410	202	303	2026-10-06	Scheduled	[null]

Conclusion

The GreenCycle database successfully implements a structured system for managing households, routes, vehicles, waste types, and collection schedules. The use of normalization, SQL queries, and a trigger helps reduce redundancy and automate future collection scheduling, making the database efficient and reliable.